

WHY SYSTEM CALLS LOOK LIKE PROCEDURE CALLS

You may wonder why a system call, such as `open()` or `read()`, looks exactly like a typical procedure call in C; that is, if it looks just like a procedure call, how does the system know it's a system call, and do all the right stuff? The simple reason: it is a procedure call, but hidden inside that procedure call is the famous trap instruction. More specifically, when you call `open()` (for example), you are executing a procedure call into the C library. Therein, whether for `open()` or any of the other system calls provided, the library uses an agreed-upon calling convention with the kernel to put the arguments to `open()` in well-known locations (e.g., on the stack, or in specific registers), puts the system-call number into a well-known location as well (again, onto the stack or a register), and then executes the aforementioned trap instruction. The code in the library after the trap unpacks return values and returns control to the program that issued the system call. Thus, the parts of the C library that make system calls are hand-coded in assembly, as they need to carefully follow convention in order to process arguments and return values correctly, as well as execute the hardware-specific trap instruction.

USE PROTECTED CONTROL TRANSFER

The hardware assists the OS by providing different modes of execution. In user mode, applications do not have full access to hardware resources. In kernel mode, the OS has access to the full resources of the machine. Special instructions to trap into the kernel and return-from-trap back to user-mode programs are also provided, as well as instructions that allow the OS to tell the hardware where the trap table resides in memory.

The hardware needs to be a bit careful when executing a trap, in that it must make sure to save enough of the caller's registers in order to be able to return correctly when the OS issues the return-from-trap instruction. On x86, for example, the processor will push the program counter, flags, and a few other registers onto a per-process kernel stack; the return-from-trap will pop these values off the stack and resume execution of the usermode program.

Interrupt Descriptor Table (IDT): gateway to handlers

- Interrupt descriptor table provides a way to define handlers for different events like external interrupts, faults and system calls by defining the descriptors
- Descriptors 0-31 are for predefined events e.g., 0 → Div-by-zero exception etc. - Events 32-255 are user defined, can be used for h/w and s/w interrupt handling

How CPU Uses the IDT

- An event occurs (exception, interrupt, syscall).
- CPU looks up the corresponding IDT entry using the event number.
- CPU reads the descriptor → gets handler address, segment selector, privilege info.
- CPU pushes current context onto the stack (EIP/RIP, CS, EFLAGS, maybe error code).
- CPU jumps to the handler.
- Handler executes → issues `iret` to return to original execution.

Entries 0–31:

- Reserved for CPU-defined exceptions.

-Examples:

`#DE` (0) → Divide-by-zero, `#PF` (14) → Page Fault, `#GP` (13) → General Protection Fault,
`#DF` (8) → Double Fault.

Entries 32–255:

- User-defined / OS-defined.

- Can be used for:

Hardware interrupts (e.g., keyboard, timer, NIC).

Software interrupts (system calls via `int 0x80` or `syscall` in Linux).

What is the IDT?

- The Interrupt Descriptor Table (IDT) is a data structure used by the x86/x86-64 CPU to define how to handle interrupts, exceptions, and system calls.

- Each entry in the table is called an IDT Descriptor, which contains the address of the handler function and privilege-level information.

- The CPU uses the IDTR (Interrupt Descriptor Table Register) to know where the IDT is located in memory.

Structure of IDT

Size: 256 entries (0–255).

Each entry = a descriptor (8–16 bytes depending on mode).

Fields inside each descriptor:

- Offset (handler address): 64-bit pointer to the ISR/handler.

- Selector: Code segment selector from GDT/LDT.

- Attributes/Type: Defines gate type (interrupt gate, trap gate, task gate), privilege level.

- Reserved bits.

```
asm volatile("mov %%cr3, %0;" : "=r" (cr3_val));
```

User executes "mov %cr3"

↓

CPU privilege check fails

↓

CPU raises #GP(13)

↓

CPU looks up IDT[13] → handler entry

↓

Switches to kernel mode, runs handler kernel's `do_general_protection()`

↓

Linux kernel decides → send SIGSEGV → kill process

- INT #N: Raise a software interrupt. CPU invokes the handler defined in the IDT descriptor #N (if registered by the OS)

- Conventionally, IDT descriptor 128 (0x80) is used to define system call entry gates

- The generic system call handler invokes the appropriate handler function, How?

- Every system call is associated with a number (defined by OS)

- User process sends information like system call number, arguments through CPU registers which is used to invoke the actual handler

To run a secure OS with user/kernel separation, the CPU must provide mechanisms for:

1. Privilege Levels

x86 uses 4 rings (0–3):

- Ring 0 (kernel mode): full privileges → can execute any instruction, access all memory, configure hardware.
- Ring 3 (user mode): restricted → cannot access privileged registers (e.g., CR3), cannot directly touch devices, cannot modify page tables.

2. Mode Switching

- Hardware must allow controlled transitions from user mode (Ring 3) → kernel mode (Ring 0).
- Entry points are well-defined by the OS:
 - Exceptions (e.g., divide-by-zero, page fault).
 - Interrupts (e.g., keyboard input, timer tick).
 - System calls (software-invoked entry point, e.g., int 0x80 or syscall).

When such an event occurs:

- CPU automatically switches to Ring 0.
- CPU pushes a stack frame with old RIP, CS, RFLAGS, etc.
- CPU jumps to the handler address from the IDT.

3. Interrupt Descriptor Table (IDT)

- CPU uses the IDT to know which handler to run for each event.
- Each IDT entry specifies:
 - Handler address (where kernel code begins).
 - Privilege rules (who can call it — e.g., syscall gate has DPL=3).
 - Type (interrupt gate vs trap gate).
- OS fills the IDT with its handler addresses and loads it via LIDT.

4. System Call Entry Support

- int N software interrupt (e.g., int 0x80) : CPU looks at IDT[N] → jumps to handler.

5. Hardware Enforcement

- If user code tries to:
 - Access a privileged register (e.g., mov %cr3 in Ring 3).
 - Access memory without permission.
 - Jump directly to kernel code.
 - CPU raises a fault/exception (#GP, #PF).
 - Kernel handler (via IDT) runs and typically kills the offending process.

This hardware enforcement is what prevents user-space from bypassing the OS.

To support secure OS operation, x86 hardware provides:

1. Privilege levels (rings) → user vs kernel separation.
2. Controlled entry points → exceptions, interrupts, syscalls.
3. Interrupt Descriptor Table (IDT) → maps events to kernel handlers.
4. Syscall support instructions (int N, syscall/sysret).
5. Automatic stack/context switching → CPU pushes RIP, CS, RFLAGS, etc. on privilege change.

The OS defines the handlers and entry point behavior (through IDT and syscall table), while the CPU enforces privilege checks and safe switching.

Example:

```
... //Rb = num Ra =flag
```

```
0x50000: callq sqrt //R1 = sqrt (num)
```

```
0x50004: mov $2, R2 //R2 = i
```

```
0x50008: cmp R1, R2
```

```
0x5000c: jge $0x50024
```

```
0x50010: div Rb, R2 // Reminder in R2
```

```
0x50014: incr R2
```

```
0x50018: cmp R3, $0
```

```
0x5001c: jne $0x50008
```

```
0x50020: mov $0, Ra
```

```
0x50024: ret // Return Ra
```

Let us assume that interrupt fires at PC = 0x50018, At OS entry State : Rb=97, Ra=1, R2=7, R3=3 (user code mid-loop)

When an interrupt occurs (could be timer, external device, or exception):

CPU switches to kernel mode and jumps to `os_handler`.

In the handler:

-It clobbers registers (Ra, R2, R3) without saving user state.

-Eventually it executes `iret` to return to the user.

Return To User State (after handler and `iret`):

`Rb=-`, `Ra=X`, `R2=1`, `R3=0`, `PC=0xF00200` (values clobbered by OS code).

Problem: The OS handler overwrote registers without saving/restoring. So when control returns, user code sees **garbage in its registers**, making the algorithm fail.

What causes the mode switch?

- Exceptions (divide error, page fault, etc.)
- External interrupts (timer tick, device IRQs)
- Software interrupts (if user code executes `int n`)

In this example, assume a timer interrupt fired while the loop was running.

What went wrong?

- The OS handler didn't save/restore general-purpose registers before using them.
- When `iret` executed, only PC/flags/segments were restored, not the working registers.

Correct behavior:

- On entry: save all registers (push to stack or build a `pt_regs` frame).
- Do handler work.
- Restore registers.
- `iret` → return cleanly, user state unchanged.

Management of OS stacks

- A per-process OS stack is required to allow multiple processes to be in OS mode of execution simultaneously
- Working : The OS configures the kernel stack address of the currently executing process in the hardware, the hardware switches the stack pointer on system call or exception
- What about external interrupts? : Separate interrupt stacks are used by OS for handling interrupts

Syscall/Trap handler → scheduler

- Happens when the process voluntarily enters the kernel.
- Example: `read()` system call.
- Before returning, kernel can decide: If the process used up its CPU quota, call scheduler, if it's waiting on I/O, put it to sleep and schedule another.

Interrupt handler → scheduler

- This is asynchronous (process didn't ask for it).
- Example: timer interrupt.
- OS can configure periodic timer interrupts to regain control regularly.
- After handling interrupt, OS can call the scheduler to enforce fairness (preemption).

Key Idea:

Every time the CPU traps into the kernel (via a syscall, exception, or interrupt), the OS has an opportunity to invoke the scheduler.

Flow:

1. User process runs → makes a syscall (e.g., read()).
2. CPU switches to kernel → executes syscall handler.
3. Handler may realize: "This process is waiting for I/O, it cannot proceed."
→ Instead of keeping it on CPU, the kernel marks it as blocked.
4. Control passes to the scheduler.
 - Scheduler picks another runnable process.
 - Original process waits until the I/O completes and will be rescheduled later.

Extra Notes:

- Condition mostly arises in I/O-related syscalls (like reading from disk).
- Process is put into wait state → CPU is given to another process

OS switches from P1 → P2:

1. Enter kernel (syscall/interrupt).
2. Kernel runs on P1's kernel stack.
3. Scheduler chooses P2.
4. Save P1's registers/stack into PCB.
5. Restore P2's registers/stack from PCB.
6. Now OS runs on P2's kernel stack.
7. Exit to user mode → P2 continues.

Scheduler flow:

New Process Creation : When a new process is created, it is placed in the Ready Queue.

Scheduling a Process : OS Scheduler picks one process from the Ready Queue. -> That process is dispatched to the CPU.

CPU Execution : Process runs on CPU until one of the following happens:

- Exit → finishes execution, leaves system.
- Deschedule → preempted (time slice expired, higher priority task arrived). Returns to Ready Queue.
- Wait for event → makes a blocking call (e.g., read() from disk). Goes to Wait Queue.

Waiting & Event Completion :

-A process in the Wait Queue stays idle until the event completes (disk I/O done, network packet arrives, etc.).

-Once the event finishes, the process moves back to the Ready Queue.

Repeat

- Scheduler continues choosing processes from Ready Queue.
- Ensures fairness, efficiency, and responsiveness.

System idle process: There can be an instance when there are zero processes in ready queue

A special process (system idle process) is always there : The system idle process halts the CPU it calls the HLT instruction on X86_64: Halts the CPU till next interrupt

Types of Scheduling :

There are scheduling points which are triggered because of the current process execution behavior (non-preemptive) - Process termination - Process explicitly yields the CPU - Process waits/blocks for an I/O or event

The OS may invoke the scheduler in other conditions (preemptive) - Return from system call (specifically fork()) - After handling an interrupt (specifically timer interrupt)

Scheduling metrics ->

Turnaround time: Time of completion - Time of arrival

-Objective: Minimize turnaround time

Waiting time: Sum of time spent in ready queue

- Objective: Minimize waiting time

Response time: Waiting time before first execution

- Objective: Minimize response time

Average value of above metrics represent the average efficiency

Standard deviation represents fairness across different processes

Note: The OS address space consumes a part of the address space from all processes and protect the OS addresses using H/W assistance

Turnaround Time(TAT) : Completion time - Arrival Time

Waiting Time(WT) : TAT - Burst Time

Response Time : First execution time - Arrival Time

Pseudocode: Round Robin Scheduling

Input:

processes[] = list of (pid, arrival_time, burst_time)

time_quantum

Output:

Completion time, Turnaround time, Waiting time for each process

Algorithm:

1. Initialize:

- ready_queue = empty queue
- current_time = 0
- remaining_time[pid] = burst_time for each process
- completion_time[pid] = 0

2. Sort processes by arrival_time.

3. While (some process still incomplete):

a. Add all processes that have arrival_time ≤ current_time
and are not yet in the queue to ready_queue.

b. If ready_queue is empty:

current_time = next arrival_time
continue

c. Dequeue process P from ready_queue.

d. If remaining_time[P] > time_quantum:

- Execute P for 'time_quantum'
- current_time += time_quantum
- remaining_time[P] -= time_quantum
- Add newly arrived processes (arrival_time ≤ current_time)
to ready_queue.
- Enqueue P back to ready_queue.

Else (remaining_time[P] <= time_quantum):

- Execute P for 'remaining_time[P]'
- current_time += remaining_time[P]
- remaining_time[P] = 0
- completion_time[P] = current_time
- Process P is finished (don't re-enqueue).

4. After all processes complete:

For each process P:

turnaround_time[P] = completion_time[P] - arrival_time[P]

waiting_time[P] = turnaround_time[P] - burst_time[P]

5. Return all completion_time, TAT, WT.

First Come First Served (FCFS) - FIFO queue based non-preemptive scheduling - Example - Advantages - Easy to implement - Issues with FCFS - Convoy effect - Not suitable for interactive applications

Shortest Job First (SJF) - Select the process with shortest CPU burst - Pick the next process only when the current process is finished (non-preemptive) - Example - Optimal on waiting time and turnaround time - Not realistic (how can we know the execution time?)

Shortest Time to Completion First (STCF) - Pick the process with shortest remaining time when a new process arrives in the ready queue (SRTF) - Example - Improves the efficiency of SJF at the cost of more context switches

Round-robin scheduling - Preemptive scheduling with time slicing - Ready queue is maintained as a circular queue - At end of the time quantum, If there are other processes in the queue - Current process goes to the TAIL of the queue - Next process is picked up from the HEAD of the queue - New processes are added to the TAIL of the queue - Design choice: size of time quantum

Priority scheduling - Select the process with highest priority - Can be preemptive and non-preemptive - SJF: priority defined by job length - Advantages: practical (no assumptions) - Disadvantages: Starvation

Most processes require a series of CPU and I/O bursts - Looks complicated for analysis, can it be simplified? - Every CPU burst is treated as a new process where the CPU burst start is the process arrival time and burst length is the execution time

Multilevel feedback queue -

Basic multi-level strategy - Pick a process from highest priority queue - Within a queue, apply RR

Dynamically adjust priorities such that 1. Interactive applications are responsive 2. Short jobs do not suffer 3. No starvation 4. No user can trick the scheduler

- A process is assigned the highest priority when it is created - If the process consumes the slice (scheduler invoked because of timer), its priority is reduced - If the process relinquishes the CPU (I/O wait etc.), its priority remain the same

MLFQ: Approximation of SJF - MLFQ can approximate SJF because - Long running jobs are moved to low priority queues - New jobs are added to highest priority queue - A shorter job may not get a chance to execute for a small duration. What is the upper bound?

- (# of jobs in the highest priority queue at arrival + 1) X (time quantum)

MLFQ: Interactive jobs - MLFQ favors interactive jobs because - Interactive jobs maintain the highest priority as they relinquish the CPU before quantum expires - Long running jobs are moved to low priority queues - Conclusion: In a steady state, interactive jobs compete with short and other interactive jobs

Pseudocode: MLFQ (with RR inside each queue)

Input:

processes[] = list of (pid, arrival_time, burst_time)

Q[1..k] = ready queues (priority 1 is highest)

time_quantum[1..k] = quantum length per queue

Rules:

- New process → goes into Q[1]
- If process uses full quantum → demote to next lower queue
- If process blocks (I/O) or voluntarily releases CPU → keep same priority
- Scheduler always runs the highest non-empty queue
- Within each queue → use Round Robin

Algorithm:

1. Initialize:

current_time = 0

remaining_time[pid] = burst_time for each process

first_start_time[pid] = -1

completion_time[pid] = 0

All queues Q[i] = empty

2. While (some process still incomplete):

a. Add newly arrived processes ($\text{arrival_time} \leq \text{current_time}$)

into $Q[1]$.

b. If all queues are empty:

$\text{current_time} = \text{next arrival_time}$

enqueue that process in $Q[1]$

continue

c. Find the highest priority queue $Q[i]$ that is not empty.

d. Dequeue process P from $Q[i]$.

e. If $\text{first_start_time}[P] == -1$:

$\text{first_start_time}[P] = \text{current_time}$ // for Response Time

f. Run P :

If $\text{remaining_time}[P] > \text{time_quantum}[i]$:

- Execute P for $\text{time_quantum}[i]$

- $\text{current_time} += \text{time_quantum}[i]$

- $\text{remaining_time}[P] -= \text{time_quantum}[i]$

- Add new arrivals ($\text{arrival_time} \leq \text{current_time}$) to $Q[1]$

- Demote P to $Q[i+1]$ if $i < k$

- If $i == k$, enqueue back into $Q[k]$ (stay in lowest queue)

Else ($\text{remaining_time}[P] \leq \text{time_quantum}[i]$):

- Execute P for $\text{remaining_time}[P]$

- $\text{current_time} += \text{remaining_time}[P]$

- $\text{remaining_time}[P] = 0$

- $\text{completion_time}[P] = \text{current_time}$

- Process P is finished (not re-enqueued)

3. After all processes complete:

For each process P :

$\text{turnaround_time}[P] = \text{completion_time}[P] - \text{arrival_time}[P]$

$\text{waiting_time}[P] = \text{turnaround_time}[P] - \text{burst_time}[P]$

$\text{response_time}[P] = \text{first_start_time}[P] - \text{arrival_time}[P]$

4. Return all TAT, WT, RT and their averages/standard deviations.

What is a Time Quantum?

- The time quantum (sometimes called time slice) is the maximum amount of CPU time a process is allowed to run before the scheduler forces a context switch.

OS- Kernel context switch

- The OS configures the kernel stack of the process before scheduling the process on the CPU
- The CPU saves the execution state onto the kernel stack
- The OS handler finds the SP switched with user state saved
- The OS executes the event (syscall/interrupt) handler
- Makes use of the kernel stack
- Execution state on CPU is of OS at this point
- The kernel stack pointer should point to the position at the time of entry
- CPU loads the user execution state and resumes user execution
- **Which address space the OS uses?**
- Consume a part of the address space from all processes and protect the OS addresses using H/W assistance
- A part of the process address space is reserved for OS and is protected

Process : context switch

Interrupt/syscall occurs while CPU is in user mode.

CPU switches to **kernel mode** and saves user registers + PC onto the process's **kernel stack**.

OS handler runs in kernel mode.

Scheduler is called to decide the next process.

Save current process's kernel context into its PCB.

Load new process's kernel context from its PCB.

Switch stack pointer (SP) to new process's kernel stack.

Restore registers and program counter for the new process.

Execute `iret` (or similar) → return to **user mode** of the new process.